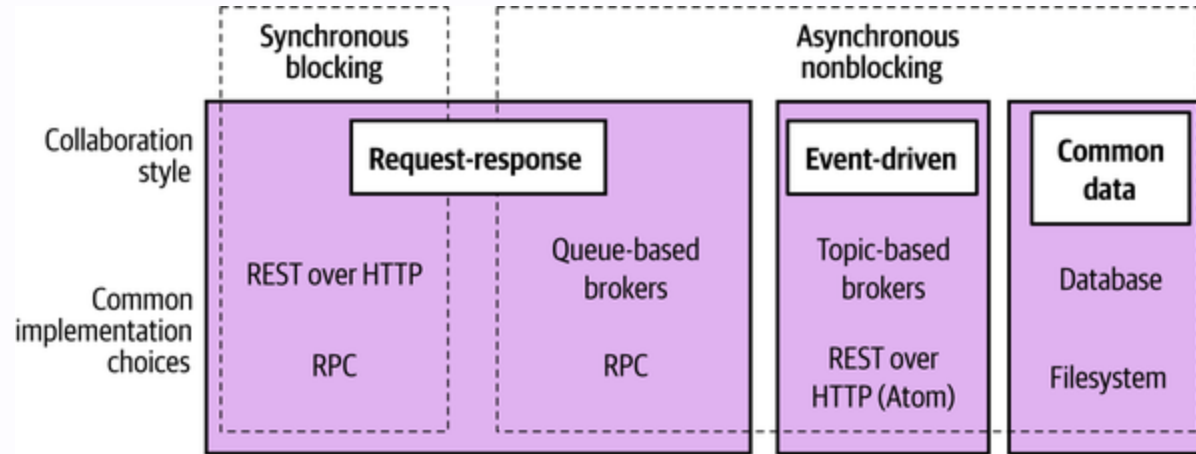


Unidad 4

Implementación

De la unidad de pasada...



Estilo ⇨ Patrón ⇨ Tecnología

Ej. Sincrónico con bloqueo ⇨ *request-response* ⇨
REST



Criterios para elegir tecnología

- Facilitar la compatibilidad con versiones anteriores: consumidores que usan versiones antiguas de la API deben seguir funcionando.
- Interfaz explícita: se debe exponer claramente sus funcionalidades y qué hacen, tanto para los desarrolladores como para quienes la consumen.

- Simple para los consumidores: buena documentación, intuitivo y compatible con distintas tecnologías.
- Ocultar detalles de implementación interna: más deben saber los consumidores de un servicio, más aumenta el acoplamiento.



Elección de tecnología

Existen múltiples tecnologías de comunicación entre servicios. Para efectos de este curso, veremos:
Remote Procedure Calls (RPC), REST, GraphQL y Message brokers.

Remote Procedure Calls (RPC)

La *Remote Procedure Calls* (RPC) se refiere a hacer una llamada local y hacer que se ejecute en un servicio remoto. Hay diferentes implementaciones de RPC.

Por lo general, usar una tecnología RPC incluye un protocolo de serialización.

Las implementaciones más conocidas son XML-RPC, SOAP y gRPC.

Ejemplo de servidor RPC

```
from xmlrpc.server import *

class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

server = SimpleXMLRPCServer(('localhost', 8000),
                             requestHandler=RequestHandler)
server.register_introspection_functions()

@server.register_function
def add(x, y):
    return x + y

server.serve_forever()
```

Ejemplo de cliente RPC

```
import xmlrpc.client

s = xmlrpc.client.ServerProxy('http://localhost:8000')
print(s.add(2,3)) # Returns 5
```

Ejemplo de esquema gRPC

```
syntax = "proto3";

service Calculator {
    rpc Add (AddRequest) returns (AddReply);
}

message AddRequest {
    int32 x = 1;
    int32 y = 2;
}

message AddReply {
    int32 result = 1;
}
```

Hoy gRPC es la implementación RPC dominante: el esquema `.proto` define el servicio y los mensajes se serializan en binario con Protocol Buffers.

Desventajas de RPC

- Acoplamiento tecnológico: algunas implementaciones de RPC están fuertemente vinculadas a una plataforma o lenguaje específico.
- Las llamadas locales no son llamadas remotas: la idea central de RPC es ocultar la complejidad de una llamada remota. Sin embargo, puede esconderse demasiado.
- Es frágil: a cambios y extensiones en las funciones del servidor.

{ } REST sobre HTTP

- Representational State Transfer (REST) es un patrón arquitectónico inspirado en la web.
- REST se basa en el concepto de recursos. Un recurso es una entidad que el servicio maneja (como los productos o cliente).
- Habitualmente se representan estos recursos como un JSON.
- Es normal exponer la URI de recursos relacionados.

- REST utiliza las capacidades de HTTP, en particular los verbos (GET, POST, PUT y DELETE).
 - Hay muchos ejercicios para estandarizar (y documentar mejor) REST, como [OpenAPI](#).
 - Lectura recomendada: [Best Practices for Designing a Pragmatic RESTful API](#).
-  `demo_01` de la Unidad 1 es un buen ejemplo de una API REST.

Desventajas de REST

- Es más difícil de consumir una API REST que una RPC. Hay una tendencia a crear librerías para estas API (lo cual aumenta el acoplamiento).
- Cuesta encontrar API REST bien documentadas, OpenAPI no es (aún) tan fácil de implementar. Aun así, más propicia a ser bien documentada que RPC.
- Tienen menor rendimiento que RPC.

GraphQL

- Una consulta puede traer (fácilmente) múltiples entidades.
- Es el cliente el que determina los datos que requiere del servidor.
- Existen dos tipos de consulta, las *query* (obtener datos) y las *mutations* (modificar datos).

Lectura recomendada: [Using GraphQL with Python – A Complete Guide.](#)

Desventajas de GraphQL

- Aunque han habido avances significativos, dependes de que GraphQL esté implementado en tu lenguaje.
- Es fácil realizar consultas poco óptimas.
- Su forma de realizar consultas, similar a SQL, refuerza la idea (equivocada) que los microservicios son *wrapper* de bases de datos.

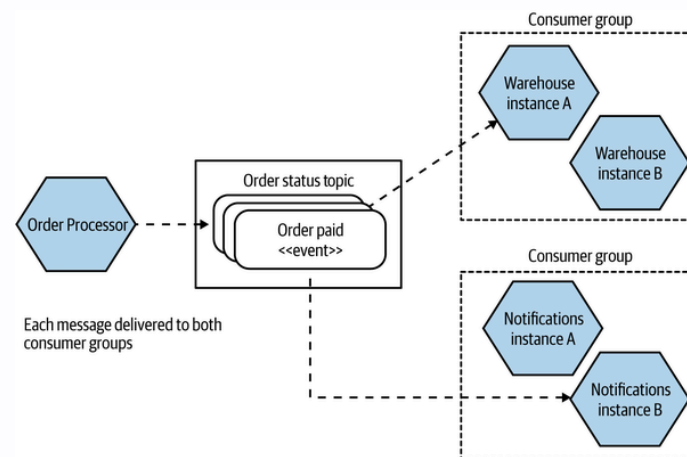
Message brokers

- *Message brokers* son intermediarios (*middleware*), que manejan la comunicación entre procesos (o servicios).
- Son una opción popular para manejar comunicación asíncrona entre servicios.
- Un mensaje es algo que un *message brokers* manda. Puede contener una consulta, una respuesta o un evento.

- Un servicio envía el mensaje al *message brokers* sin tener conocimiento de que servicios lo recibirán.
- Existen múltiples *message brokers*, por ejemplo [RabbitMQ](#), [Redis](#) o [Apache Kafka](#).
- Nubes como Google Cloud, Amazon y Azure proveen sus propios *message brokers*.
- Para efectos de este curso, usaremos como *message brokers* [RabbitMQ](#).

Tópicos y colas

- Habitualmente los *message brokers* implementan sistemas de tópicos, colas o ambos.
- En **RabbitMQ** un mensaje es enviado a un tópico (*exchange*), una cola (*queue*) se suscribe a uno o más tópicos.
- Los suscriptores (replicas de un microservicio en nuestro caso) toman cada mensaje de la cola y lo procesan.



Entrega garantizada

- Los *message brokers* garantizan que los mensajes son entregados al tópicos y a las colas que lo suscriban.
- Si el servicio receptor está inaccesible, llegará cuando vuelva a estar disponible.
- No se garantiza que el resultado del procesamiento del mensaje haya resultado con éxito. Tampoco se garantiza el orden en que se procesan.



Formatos de serialización

- Algunas de las tecnologías que hemos conversado vienen con formatos de serialización.
- En el caso que no, se debe elegir entre formatos textuales (como JSON y XML) o binarios (como Protocol Buffers, Avro o MessagePack).



Esquema

- En este contexto, es qué expone y qué acepta en cada *end-point* un microservicio.
- Varias de las tecnologías que hemos revisado requieren definir esquema. Siendo o no requerido, es recomendable.
- Ayudan a capturar *breaking change*. Hay de dos tipos: estructurales y semánticos.

- Un cambio estructural hace que la forma que se usaba antes el *end-point* ya no funciona: cambia la cantidad de parámetros requeridos, cómo se llama o la estructura de lo que devuelve.
- Un cambio semántico es cuando la forma a la que se accede a un *end-point* se mantiene pero el comportamiento de este cambia. Esto es más peligroso.



Versionamiento semántico

MAYOR.MINOR.PATCH

- Mayor: cambios drásticos no compatibles con versiones anteriores.
- Minor: agregar o extender una funcionalidad. Es compatible con versiones anteriores.
- Patch: corrección de algún error o mejoras menores. Es compatible con versiones anteriores.



Evitar los *breaking change*

- Extiende la interfaz (API), no elimines cosas viejas.
- Consume la interfaz pensando en que puede cambiar. Sé flexible.
- Usa tecnologías que permitan compatibilidad con cambios.
- Interfaces explícitas en qué siempre contienen y qué puede cambiar.
- Captura los *breaking change* accidentales antes que ocurran (pruebas automáticas idealmente).

Ejemplo de errores a evitar

```
/* Versión original */  
{  
  "firstname": "Rafik",  
  "lastname": "Mas'ad",  
  "email": "rmn@nursoft.co",  
}
```

```
/* Nueva versión */  
{  
  "naming": {  
    "firstname": "Rafik",  
    "lastname": "Mas'ad",  
    "nickname": "RMN"  
  },  
  "number": "+569-1234-567"  
}
```

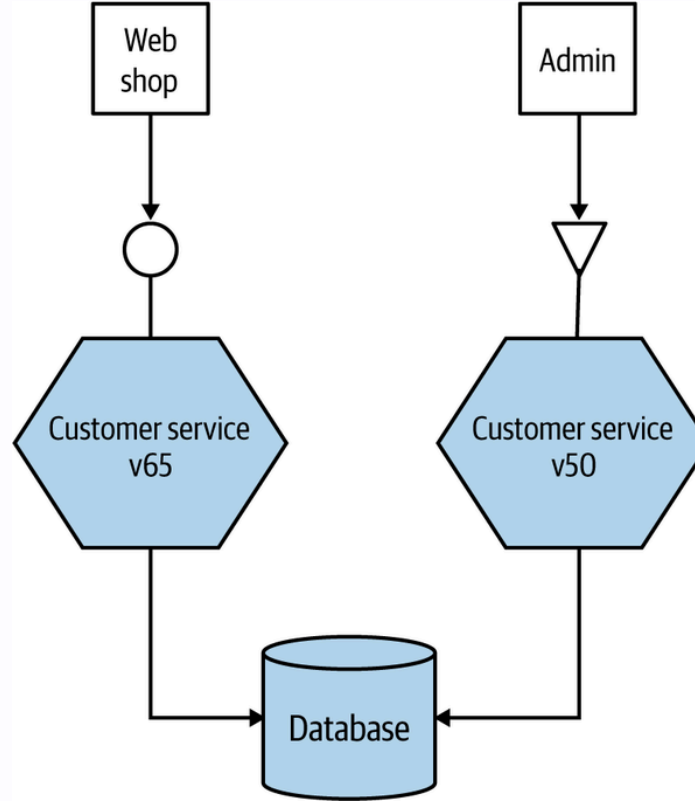
Los errores cometidos son:

- No se extiende la interfaz, al agregar dentro de `naming` parámetros anteriores ya no es compatible con la versión anterior.
- Se borra el correo electrónico: si se estaba usando, se rompe el cliente.
- Se agrega el parámetro `number`, el cual no es explícito (mejor `phone_number`).

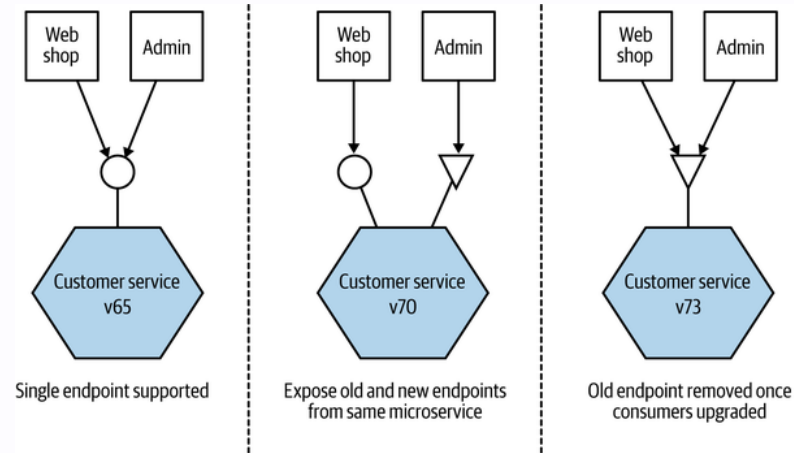
Manejar los *breaking change*

Si no es posible evitar los *breaking change*, entonces:

- Que el microservicio que exponga la interfaz y todos los consumidores de esa interfaz cambien al mismo tiempo.
- Ejecuta tanto las versiones antiguas como las nuevas del microservicio.
- Que el microservicio exponga la nueva interfaz y también emule la interfaz anterior.



Coexistencia de versiones incompatibles del microservicio

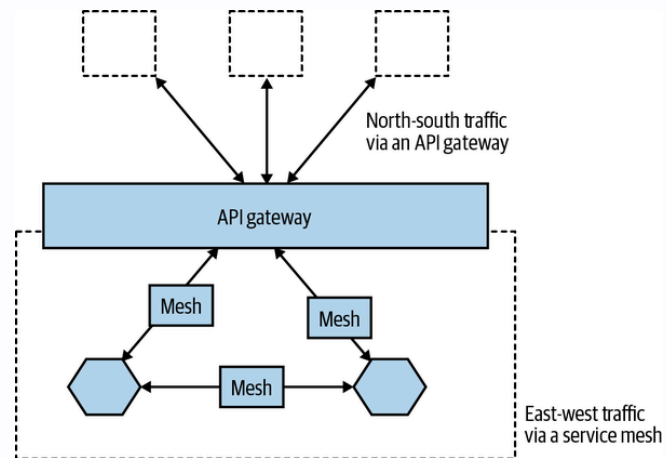


Emular la vieja interfaz

Service Meshes y API Gateways

Ambos son *middleware* (intermediarios) entre un microservicio y sus clientes (que pueden ser otros microservicios).

En la jerga de sistemas, los *services meshes* se utilizan en el tráfico este-oeste (dentro del cluster) y los *API gateways* en tráfico norte-sur (fuera del cluster).





API Gateway

En microservicios, su principal preocupación es mapear solicitudes de clientes externos a servicios internos.

Adicionalmente, los *API gateway* pueden agregar entidades de distintos servicios, implementar sistemas de *logging*, limite de consultas, etc.

Para este curso usaremos GraphQL para implementar los *API gateway*.



Service Meshes

Con una *service mesh*, funcionalidades comunes asociadas con la comunicación entre microservicio se implementan en el *mesh*.

Esto reduce las funcionalidades que un microservicio necesita implementar internamente, al tiempo que proporciona consistencia sobre cómo se hacen ciertas cosas.

Documentación

Documentar cualquier API es caro (y aburrido). Para minimizar la cantidad de documentación, podemos:

- Crear esquemas tanto de los datos de entrada que requiere un servicio como lo que retorna.
- Sistemas auto-documentables, donde el código y los comentarios sirven de documentación para los clientes de la API.

Ejemplo: `./demo_04`

- Dos servicios con API REST con [FastAPI](#).
Documentación de las APIs en `localhost:{5001-5002}/docs`.
- Comunicación asíncrona entre servicios mediante un *message broker* [RabbitMQ](#).
- *API Gateway GraphQL* desarrollado con [ariadne](#).
Incluye *queries* y una *mutation*.
- Vinculación entre recursos mediante *bindables*.
- Solución del problema del n+1 con *dataloaders*.

Transacciones

Una transacción es cuando una o más acciones se tratan como una sola unidad. Esto podría incluir eliminar datos, insertarlos o cambiarlos. En una base de datos relacional, esto podría implicar múltiples tablas actualizadas dentro de una sola transacción.



Cuando hablamos de transacciones en bases de datos, esperamos que sean atómicas, consistentes, aisladas (isolate) y durables, o ACID como acrónimo.

✳️ **Atómicas**

Asegurarse que todas las operaciones se realicen con éxito o todas fallen.

✅ **Consistentes**

Cuando se realizan cambios en nuestra base de datos, nos aseguramos de que quede en un estado válido y consistente.

Aisladas

Permite que múltiples transacciones funcionen al mismo tiempo sin interferir entre ellas.

Durables

Asegura de que una vez que se haya completado una transacción, estamos seguros de que los datos no se perderán en caso de alguna falla del sistema.

**Una transacción entre microservicios
no puede ser atómica y no es trivial
que sean aisladas.**

Transacciones distribuidas: compromiso en dos fases

La transacción se divide en dos fases:

- La fase de votación donde se consulta si es posible realizar la transacción.
- Confirmado que se puede realizar la transacción se pasa a la fase de compromiso, donde efectivamente se guardan los datos.

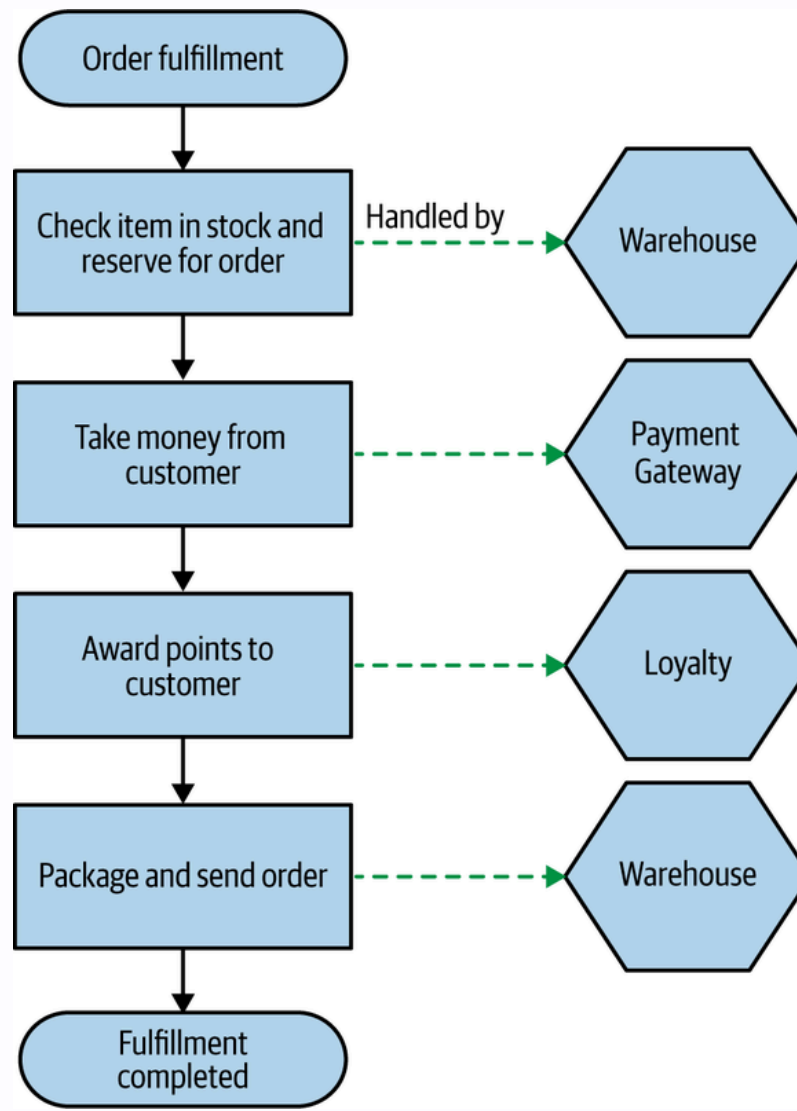
De todas formas, al ser transacciones distribuidas, no se puede asegurar que van a pasar al mismo tiempo y, mucho menos, que se van a realizar con éxito.

A veces, se implementa sistemas de *lock* para asegurarse el aislamiento de la consulta. Esto es pésima idea porque puede producir *deadlocks*.

Sagas

En 1987, Hector Garcia-Molina y Kenneth Salem definen un modelo para manejar *long lived transactions (LLTs)* (como las transacciones entre microservicios).

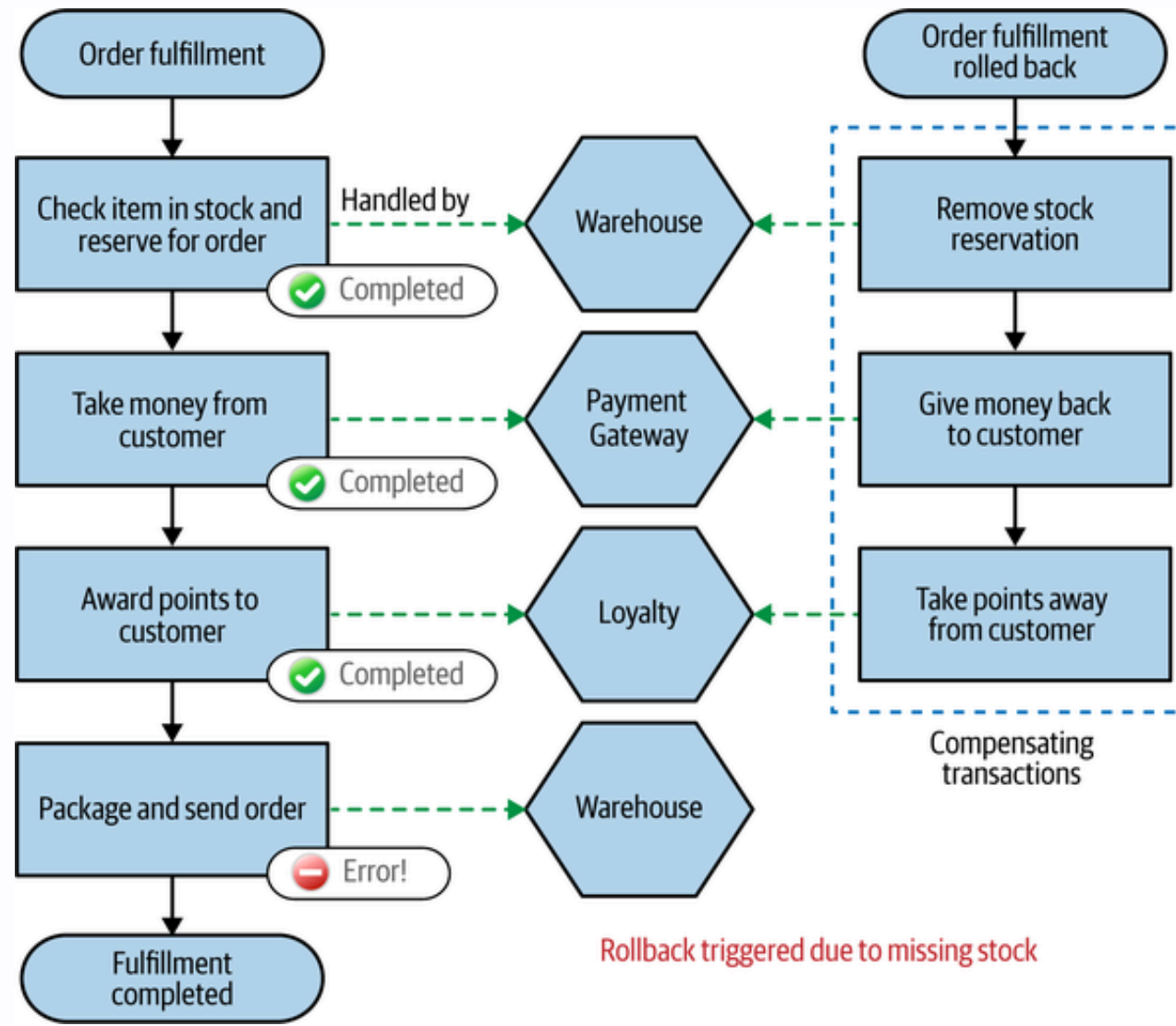
En vez de realizar compromisos en dos fases (o patrones similares), se desglosa la LLTs en etapas de negocio y se realizan pequeñas transacciones.



Modos de fallo en una Saga

- *Backward recovery*: implica revertir la transacción y limpiar lo realizado. Regresa, o intenta regresar, al punto de inicio de la transacción.
- *Forward recovery*: permite retomar desde el punto en que ocurrió la falla y seguir procesando.

Veremos más sobre sistemas resilientes en la **Unidad 8**.

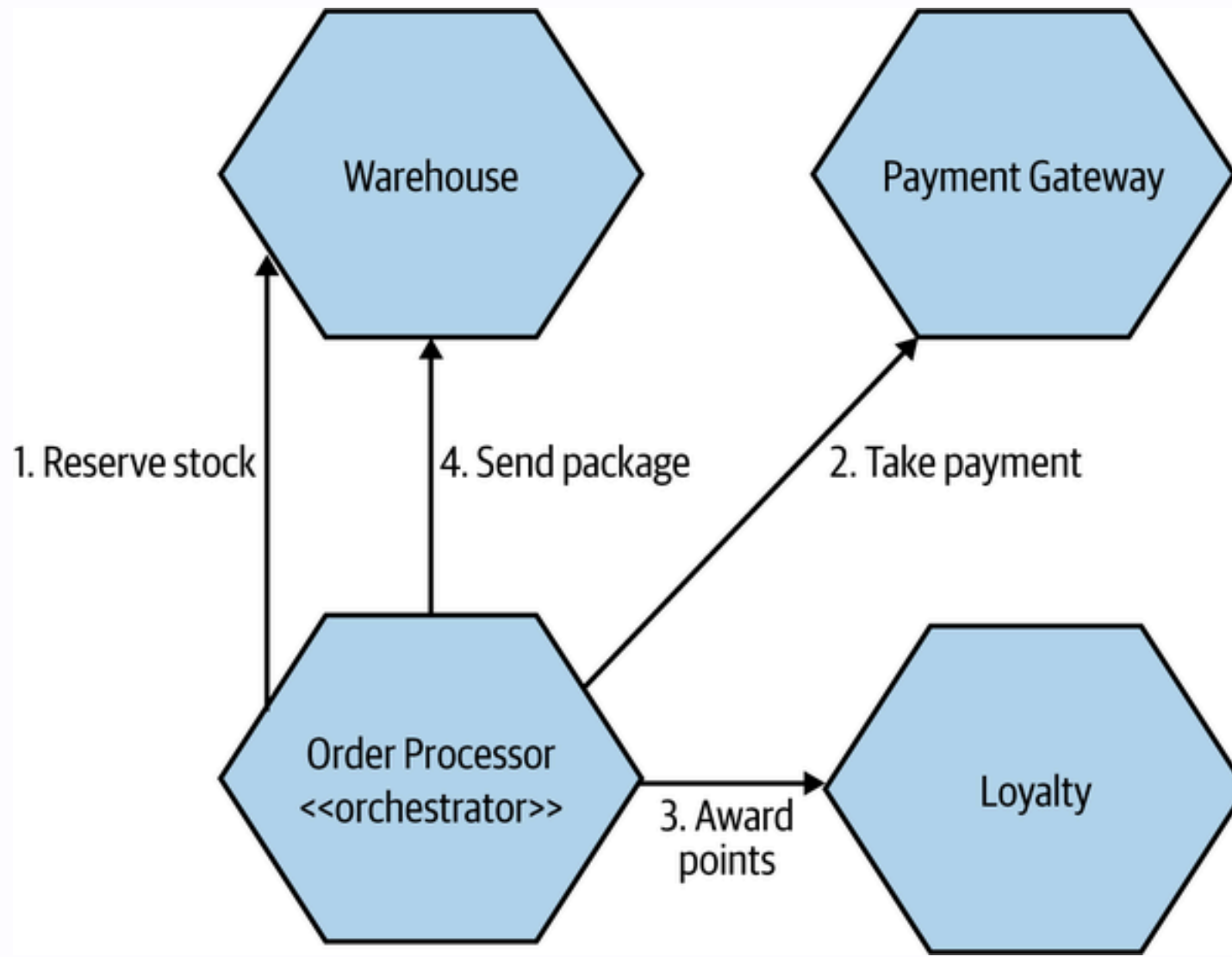


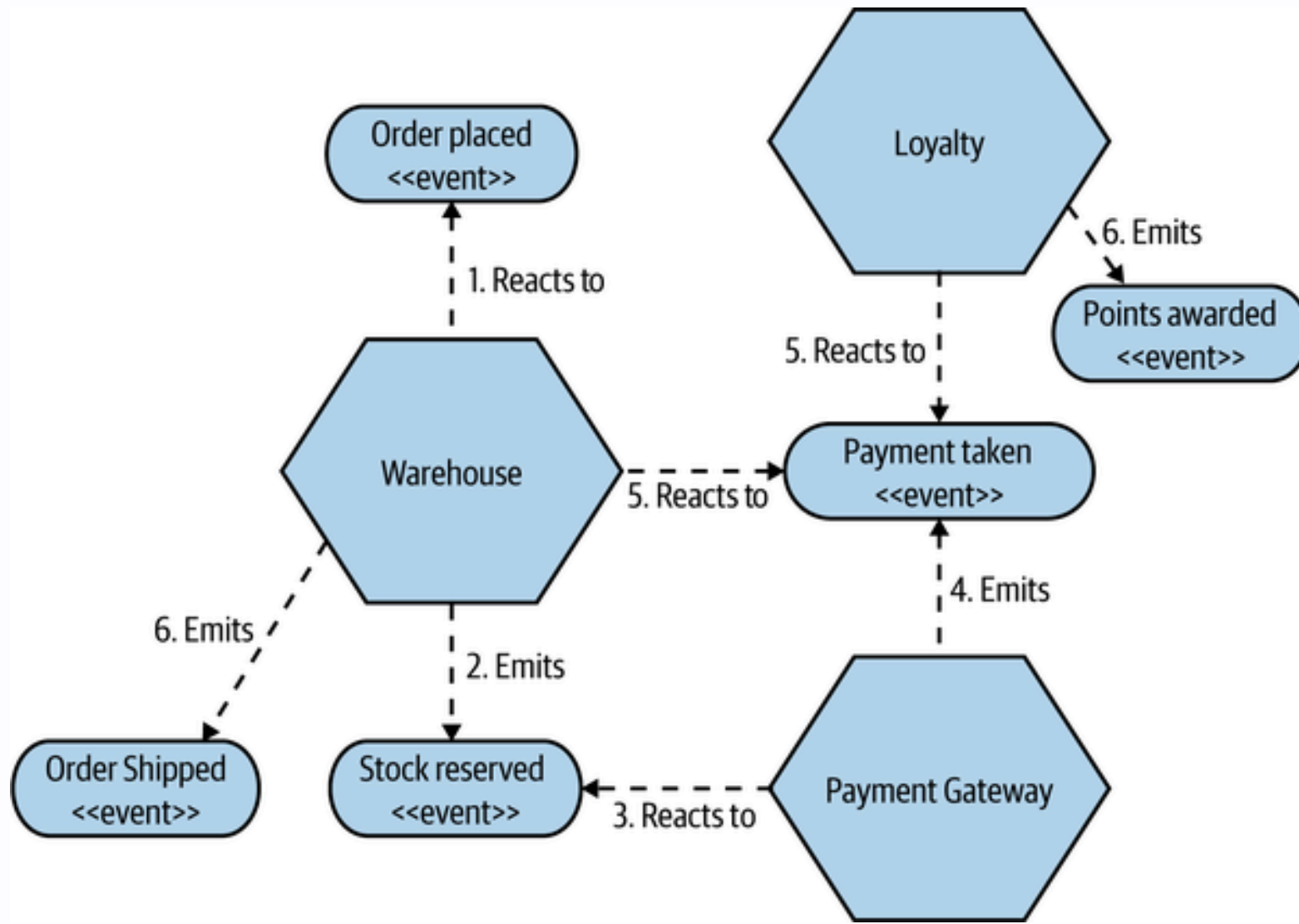


Implementando *Sagas*

Existen dos formas de implementar una *Saga*:

- Orquestación: utiliza un coordinador central (lo que llamaremos a un orquestador) para definir el orden de ejecución y activar cualquier acción requerida.
- Coreografía: se distribuye la responsabilidad del funcionamiento de la saga entre los servicios de colaboración múltiples.







Tarea

Implementa el microservicio asignado para el trabajo final. Para esto debes implementar las operaciones como una API REST y emitir eventos por cada transacción. Debe incluir documentación, esquemas, versionamiento y manejo de errores. Debe correrse su sistema mediante Docker/docker-compose.

Sube a un repositorio público el código.

Se recomienda utilizar [FastAPI](#) y leer su



Material complementario

- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capítulos 5 y 6.
- Hector Garcia-Molina y Kenneth Salem, "Sagas" ACM Sigmod Record 16, no. 3 (1987): 249–59.